

AUDIT TECHNIQUE & SÉCURITÉ

FOOTCOACH

Application PWA de coaching football — Next.js 16 / React 19 / Supabase

Périmètre	Front-end, back-end, sécurité, RGPD, accessibilité
Version auditée	V1.0.341 (branche <code>rafael-test</code>)
Base de code	77 fichiers · 13 572 lignes · TypeScript
Date	10 juillet 2026
Référentiels	OWASP · RGPD · WCAG 2.1 AA / RGAA 4.1
Fait par	Rafaël DUONG

Sommaire

1. Synthèse exécutive	3
2. Sécurité & conformité RGPD	4
3. Architecture & front-end	6
4. Accessibilité (WCAG / RGAA)	7
5. Feuille de route priorisée	8
6. Points positifs & conclusion	9

1. Synthèse exécutive

L'application est **fonctionnelle et riche**, mais l'audit révèle des **risques de sécurité critiques** incompatibles avec une mise en production, particulièrement au regard du traitement de **données personnelles de mineurs** (RGPD). L'architecture, entièrement côté navigateur, fait reposer toute la sécurité sur la configuration de la base de données (politiques RLS Supabase) — configuration aujourd'hui largement défailante.

5

FAILLES CRITIQUES

9

RISQUES ÉLEVÉS

10+

RISQUES MOYENS

3

BLOQUANTS A11Y

Verdict par domaine

Domaine	État	Constat principal
Sécurité	Critique	Escalade de privilèges, suppression de comptes arbitraire, messagerie privée lisible par des anonymes.
RGPD / données mineurs	Critique	Téléphones et données de joueurs mineurs exposés à tout compte, sans minimisation.
Architecture / Front	Élevé	100 % client-side, PWA cassée, TypeScript non strict (~98 any).
Accessibilité	Élevé	Zoom désactivé, modales inaccessibles au clavier, zéro attribut ARIA.
Qualité / Tests	Moyen	Couverture de tests quasi nulle (1 fichier), gestion d'erreurs par alert() .

La cotation de gravité combine la probabilité d'exploitation et l'impact (confidentialité, intégrité, disponibilité, conformité). Les failles « Critique » sont exploitables sans compétence particulière depuis un navigateur standard.

2. Sécurité & conformité RGPD

Toute l'application accède à la base avec la clé publique `anon` depuis le navigateur : il n'existe **aucune couche serveur d'autorisation**. Les contrôles de rôle (coach / admin) présents dans l'interface sont purement cosmétiques et contournables. La sécurité réelle dépend donc à 100 % des politiques RLS — voici leurs défaillances.

C1 — Auto-promotion en administrateur **Critique**

SCHEMA.sql:537 – table `profiles`

La politique de mise à jour du profil n'interdit pas la modification de la colonne `role`. N'importe quel utilisateur peut, depuis la console de son navigateur, se promouvoir `admin`.

Impact : compromission totale de l'application.

C2 — Suppression de n'importe quel compte **Critique**

SCHEMA.sql:454 – fonction `soft_delete_profile`

Fonction à privilèges élevés (`SECURITY DEFINER`) sans vérification de propriété et exécutable par tous. Un utilisateur peut désactiver le compte de son choix, y compris ceux des administrateurs.

Impact : déni de service ciblé, sabotage.

C3 — Suppression / fusion de clubs par tout utilisateur **Critique**

SCHEMA.sql:427 – fonction `merge_clubs`

Même faiblesse : aucune vérification de rôle côté base, seul un contrôle dans l'interface protège l'action. Un non-administrateur peut supprimer des clubs et réaffecter en masse des profils et événements.

C4 — Tables sensibles sans protection RLS **Critique**

SCHEMA.sql:522, 614 – `messages`, `audit_log`, `app_config`

La messagerie privée entre coachs est **lisible par des visiteurs anonymes** et modifiable par tout compte. Le journal d'audit (contenant des instantanés complets de profils) est falsifiable, et la configuration applicative altérable par n'importe qui.

C5 — Exposition des données personnelles de mineurs (RGPD) **Critique**

SCHEMA.sql:535 • `coach/[code]/page.tsx:51`

Tout utilisateur authentifié peut lire l'intégralité des profils, **numéros de téléphone inclus**. Sur la page publique d'un coach, le masquage du téléphone est purement visuel — la donnée transite dans la réponse réseau. Cette absence de minimisation sur des données de joueurs mineurs constitue un **manquement RGPD**.

Risques élevés

Réf.	Constat	Gravité
H4	Edge Functions sans vérification de l'appelant → envoi de notifications push arbitraires (hameçonnage) à n'importe quel utilisateur. Aucune limitation de débit.	Élevé
H5	Fonctions <code>SECURITY DEFINER</code> sans <code>search_path</code> fixé — vecteur classique de détournement.	Élevé
H3	Autorisation exclusivement côté client (rôle lu depuis <code>localStorage</code> , falsifiable) ; politiques de lecture ouvertes aux anonymes (événements, annonces, feed).	Élevé
M1	Injection de filtre dans <code>notify-sos</code> (paramètre non validé, exécuté avec la clé de service).	Moyen
M5	En-têtes de sécurité HTTP incomplets : absence de <code>Content-Security-Policy</code> et <code>HSTS</code> .	Moyen
M2/M3	Fichier <code>supabase/.temp/</code> versionné (fuite d'identifiants d'infrastructure) ; <code>.gitignore</code> à durcir.	Moyen
—	6 dépendances vulnérables détectées (dont Next.js), corrigeables par mise à jour.	Moyen

Point de vigilance : les tables `coach_connections` et la colonne `referral_code` sont utilisées par l'application mais absentes du schéma fourni — leur configuration RLS n'a pas pu être vérifiée et doit l'être impérativement.

3. Architecture & front-end

Application 100 % côté client Élevé

68 des 77 fichiers sont des composants client ; les 119 requêtes base partent toutes du navigateur via `useEffect`. Aucun rendu serveur, aucune mise en cache, aucun préchargement. Chaque page se charge vide puis déclenche ses requêtes, et l'authentification n'est pas protégée par un middleware serveur.

PWA cassée Critique

Les icônes de l'application pointent vers des CDN tiers (`icons8` , `nextjs.org`) et un dossier `public/icons` inexistant (erreurs 404). **Trois systèmes de gestion de version concurrents et incohérents** coexistent, dont un composant mort. Le service worker ne fournit aucune capacité hors-ligne réelle. L'installation en tant qu'application est donc défailante.

Thème	Constat	Gravité
TypeScript	<code>strict: false</code> et ~98 usages de <code>any</code> : les garanties de typage sont neutralisées. Types Supabase non générés.	Élevé
Données	Aucune librairie de cache (React Query / SWR). Requêtes redondantes au démarrage (session/profil récupérés 2 à 3 fois).	Élevé
Bug	Filtre du radar figé : dépendance <code>filterLevel</code> manquante dans un <code>useMemo</code> (<code>radar/page.tsx:219</code>).	Élevé
Performance	38 balises <code></code> brutes au lieu de <code>next/image</code> (optimisation configurée mais inexploitée). Animation canvas permanente (batterie).	Moyen
Composants	Fichiers surdimensionnés (jusqu'à 685 lignes) mêlant données, logique et UI. Aucune mémoïsation ; contexte global re-rendant tous les consommateurs.	Moyen
Erreurs	32 <code>alert()</code> / 5 <code>confirm()</code> comme retour utilisateur. Certaines erreurs silencieusement ignorées. Logs de debug en production.	Moyen
Tests	Un seul fichier de test (3 tests) sur 77 fichiers. Logique métier critique (radar, matchmaking, auth) non couverte.	Moyen
Nettoyage	Client Supabase mort, clés <code>localStorage</code> dupliquées (risque de désynchronisation), config ESLint absente.	Faible

4. Accessibilité — WCAG 2.1 AA / RGAA 4.1

L'interface au thème « cyberpunk » présente des **lacunes systémiques**. En l'état, la conformité légale (obligatoire pour de nombreux acteurs) n'est pas atteignable sans reprise.

Réf. WCAG	Constat	Gravité
1.4.4	Zoom désactivé (<code>user-scalable=no</code>) — échec direct, correctif trivial.	Bloquant
2.1.1 / 4.1.2	Modales inaccessibles au clavier : réécrites à la main, sans piège de focus, sans fermeture par Échap, sans rôle <code>dialog</code> — alors que des composants accessibles (Radix, vaul) sont déjà installés mais inutilisés.	Bloquant
4.1.2 / 4.1.3	Zéro attribut ARIA dans tout le code : boutons icône-seule sans libellé, chargements et erreurs non annoncés aux lecteurs d'écran.	Bloquant
2.2.2 / 2.3.1	Aucune prise en charge de <code>prefers-reduced-motion</code> ; animations infinies (pluie, glitch, scanlines) non stoppables.	Élevé
1.3.1 / 3.3.1	Champs de formulaire sans étiquette associée ; erreurs signalées uniquement par <code>alert()</code> .	Élevé
1.4.3	Contraste insuffisant : texte gris et néon sur fond noir sous le seuil 4.5:1.	Élevé
1.1.1	Logos porteurs d'information sans texte alternatif.	Moyen
2.5.8	Cibles tactiles sous 44 px (barre de navigation, boutons de fermeture).	Moyen

Point favorable : l'attribut `lang="fr"` est correctement présent et la structure des repères (`main` , `nav`) est en partie sémantique.

5. Feuille de route priorisée

Séquencement recommandé, du risque maximal au moindre effort. Les phases P0 conditionnent toute mise en production.

P0 Bloquant — avant toute mise en production

#	Action	Effort
1	Interdire la modification du rôle dans la politique de mise à jour des profils (anti-escalade).	Faible
2	Ajouter un contrôle de propriété/rôle et révoquer l'exécution publique de <code>soft_delete_profile</code> et <code>merge_clubs</code> .	Faible
3	Activer la RLS sur <code>messages</code> , <code>audit_log</code> , <code>app_config</code> et retirer l'accès en lecture anonyme global.	Moyen
4	Restreindre la lecture des téléphones/PII (conformité RGPD mineurs).	Moyen
5	Authentifier les Edge Functions, brider le contenu des notifications, ajouter une limitation de débit.	Moyen
6	Réparer la PWA : icônes hébergées localement, suppression des CDN tiers, unification du versioning.	Moyen

P1 Important — sprint suivant

#	Action
7	Fixer <code>search_path</code> sur les fonctions à privilèges ; valider les entrées des Edge Functions.
8	Ajouter les en-têtes <code>CSP</code> et <code>HSTS</code> ; durcir <code>.gitignore</code> ; mettre à jour les dépendances vulnérables.
9	Corriger le bug du filtre radar et les captures de valeurs périmées (<i>stale closures</i>).
10	Réactiver le zoom ; réécrire les modales avec les composants accessibles déjà installés.
11	Campagne ARIA (libellés, annonces) ; prise en charge de <code>prefers-reduced-motion</code> .

P2 Dette technique — à planifier

- Introduire un middleware d'authentification serveur et une librairie de cache (React Query / SWR).
- Activer `strict: true` et générer les types Supabase pour éliminer les `any`.
- Remplacer `alert()` / `confirm()` par des notifications accessibles ; migrer vers `next/image`.
- Introduire une validation d'entrée (zod) ; étendre la couverture de tests aux logiques métier.
- Contraste, étiquettes de formulaires, cibles tactiles, textes alternatifs.

6. Points positifs & conclusion

Éléments déjà solides

- Aucun secret codé en dur ; seule la clé publique est exposée côté client, la clé de service reste serveur.
- Authentification PKCE activée ; rendu JSX échappé (pas de faille XSS détectée).
- Organisation des routes claire et cohérente ; nettoyage correct des abonnements temps réel.
- Configuration d'optimisation d'images et compression déjà en place (à exploiter).
- Attribut de langue et structure sémantique partiellement conformes.

Conclusion

FOOTCOACH dispose d'une base fonctionnelle prometteuse, mais son modèle de sécurité — reposant entièrement sur des politiques de base de données aujourd'hui défaillantes — expose des données de mineurs et permet une prise de contrôle triviale. **Les six actions P0 sont prioritaires et pour la plupart rapides à mettre en œuvre** (modifications SQL ciblées) ; elles lèvent l'essentiel du risque. La reprise de l'accessibilité et de la dette technique peut ensuite être menée de façon incrémentale, sans bloquer la trajectoire produit.

Les références de fichiers et de lignes de ce rapport correspondent à la version V1.0.341 (branche `rafael-test`) et sont fournies à titre indicatif pour faciliter la remédiation.